PyTorch Basics: Tensors & GPU Execution

Department of Statistics

Oregon State University

04/29/2026

What is Pytorch

- *PyTorch* (https://pytorch.org/) is an open-source Python-based deep learning library, the most widely used since 2019
  - A replacement for NumPy to use the power of GPUs and other accelerators.
  - An automatic differentiation library (known as autograd) that is useful to implement neural networks.
  - Offers modular, flexible, and efficient building blocks (e.g. pre-trained models, loss functions, and optimizers) for designing and training deep learning models

Tensor

- A mathematical concept that generalizes vectors and matrices to potentially higher dimensions.
- From a computational perspective, tensors are multi-dimensional arrays
- PyTorch tensors are similar to NumPy arrays but with additional features for deep learning (similar API and syntax)

TENSOR BASICS

Creating Tensors

From Python data

```
torch.tensor([1, 2, 3])
torch.tensor([[1,2],[3,4]], dtype=torch.float32)
```

Zeros, Ones & Random

```
torch.zeros(3, 4)
torch.ones(2, 3)
torch.rand(3, 3)     # Uniform [0,1)
torch.randn(3, 3)    # Normal(0,1)
```

Ranges & Identity

```
torch.arange(0, 10, 2)  # [0,2,4,6,8]
torch.linspace(0, 1, 5) # 5 evenly spaced
torch.eye(3)            # 3x3 identity
```

- Tensors are multi-dimensional arrays, similar to NumPy ndarrays
- dtype controls precision (see Table in the next slide)
- Use requires_grad=True to track gradients for autograd
- GPU tensors created via device='cuda' argument

TENSOR BASICS

Tensor Data Types (dtype)

dtype	Bits	Common Use in Deep Learning
`torch.float16 / half`	16-bit	FP16 mixed-precision training, inference on GPUs with limited VRAM
`torch.bfloat16`	16-bit	Mixed precision on TPUs & modern GPUs (A100+); wider range than FP16
`torch.float32 / float`	32-bit	Default dtype — standard training, weights, activations, loss computation
`torch.float64 / double`	64-bit	Rarely used; numerical precision testing, scientific computing
`torch.int8`	8-bit	Post-training quantization (PTQ), INT8 inference acceleration
`torch.int16 / short`	16-bit	Uncommon; some audio processing or custom quantization
`torch.int32 / int`	32-bit	Indexing tensors, class labels, token IDs
`torch.int64 / long`	64-bit	Default integer — target labels, cross-entropy indices, embedding lookups
`torch.bool`	1-bit	Masks (attention masks, padding masks), conditional logic
`torch.complex64`	64-bit	Spectrograms, FFT operations, signal processing in DL

💡 Default: torch.float32 for floats, torch.int64 for integers. Use torch.set_default_dtype() to change globally.

TENSOR BASICS

Shape, Dimensionality

Shape & Dimensionality

```
x = torch.rand(2, 3, 4)
x.shape       # torch.Size([2, 3, 4])
x.ndim        # 3
x.numel()     # 24  (total elements)
x.size(0)     # 2   (first dimension)
```

TENSOR BASICS

Tensor Multiplication & Broadcasting

Element-wise (*)

```
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
c = a * b          # [4, 10, 18]
# Also: torch.mul(a, b)
```

Shapes must match or be broadcastable

Matrix Multiply (@)

```
A = torch.rand(2, 3)
B = torch.rand(3, 4)
C = A @ B          # [2, 4]
# Also: torch.matmul(A, B)
```

Inner dims must match: [m,k] @ [k,n] → [m,n]

Batched MatMul

```
A = torch.rand(8, 3, 4)  # batch=8
B = torch.rand(8, 4, 5)
C = A @ B    # [8, 3, 5]
# Also: torch.bmm(A, B)
```

Batch dims broadcast; last 2 dims follow matrix rules

Broadcasting Rules

1. Align dimensions from the right
2. Each pair must be equal or one of them is 1
3. Missing dims are treated as size 1

A shape	B shape	Result
[3, 4]	[3, 4]	[3, 4] ✓
[3, 1]	[1, 4]	[3, 4] ✓
[5]	[3, 5]	[3, 5] ✓
[2, 1, 3]	[4, 3]	[2, 4, 3] ✓
[3, 4]	[5, 4]	Error ✗

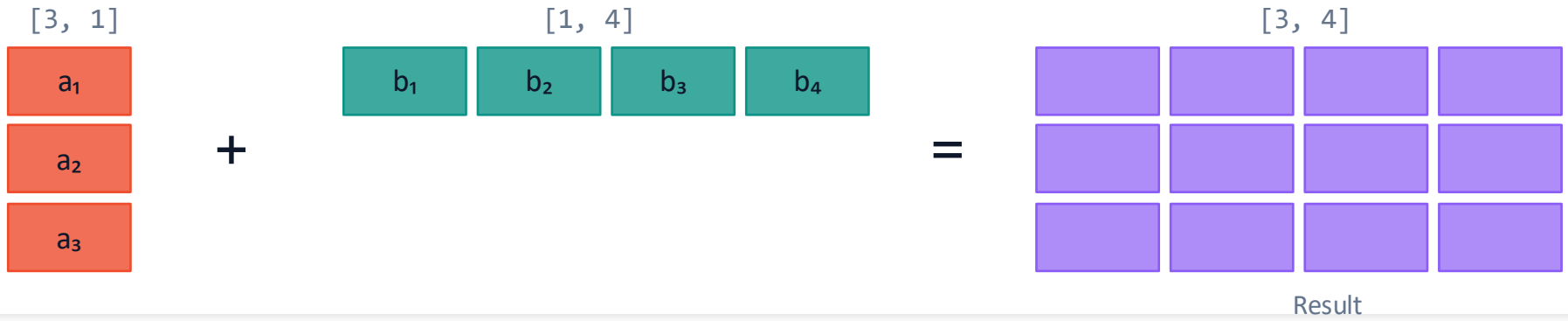
TENSOR BASICS

Broadcasting

Broadcasting Rules

```
a = torch.rand(3, 1)   # shape [3,1]
b = torch.rand(1, 4)   # shape [1,4]
c = a + b               # shape [3,4]
# Dims align right-to-left;
# size 1 stretches to match
```

Broadcasting Visualization



TENSOR BASICS

view(), reshape(), permute(), transpose()

view()

Reshapes tensor without copying data. Requires contiguous memory layout.

```
x = torch.arange(12)
y = x.view(3, 4)   # [3, 4]
z = x.view(2, -1)  # [2, 6] inferred
```

reshape()

Like view() but works on non-contiguous tensors — may copy data if needed.

```
x = torch.arange(12).view(3,4)
y = x.t()   # non-contiguous
z = y.reshape(4, 3)  # works!
# y.view(4,3) may raise error depends on version
```

permute()

Reorders all dimensions at once. Returns a view (no data copy).

```
x = torch.rand(2, 3, 4)
# [batch, height, width] -> [batch, width, height]
y = x.permute(0, 2, 1)  # [2, 4, 3]
```

transpose()

Swaps exactly two dimensions. Returns a view (no data copy). Alias: .T for 2D tensors.

```
x = torch.rand(3, 4)
y = x.transpose(0, 1)  # [4, 3]
z = x.T                # same as above
```

DEVICE TRANSFER

Moving Tensors Between Devices

CPU (Host)

```
# Default: tensors on CPU
x = torch.tensor([1.0, 2.0, 3.0])
x.device  # device(type='cpu')

# Explicit CPU placement
x = torch.rand(3, 3, device='cpu')

# Move back from GPU
x_cpu = x_gpu.cpu()
x_cpu = x_gpu.to('cpu')
```

System RAM • General computation

`.to(device)`

↔

GPU (Device)

```
# Move to GPU
x_gpu = x.cuda()
x_gpu = x.to('cuda')
x_gpu = x.to('cuda:0')  # specific GPU

# Create directly on GPU
x = torch.rand(3, 3, device='cuda')

# Check device
x.is_cuda  # True
```

VRAM • Parallel computation

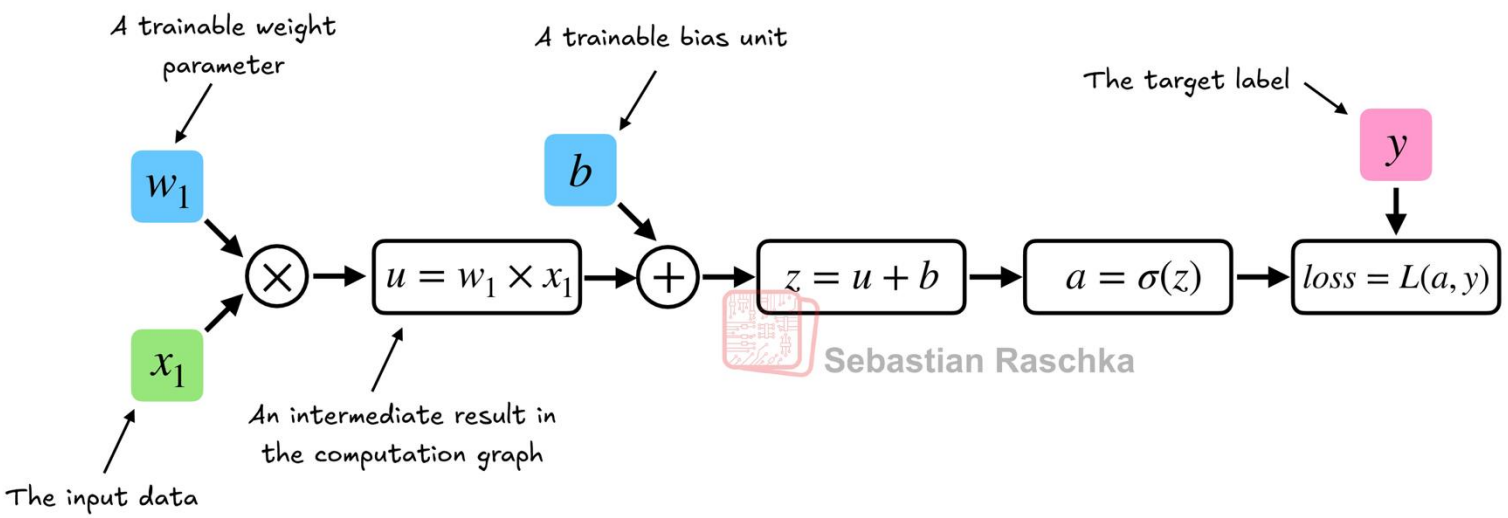
⚡ Best Practice: Minimize CPU↔GPU transfers — they are expensive. Use torch.cuda.is_available() before .cuda() calls.

Parallel computing runtime comparison of tensor manipulation

- Matrix multiplication

Computational graphs

A computational graph is a directed graph that allows us to express and visualize mathematical expressions.



Forward pass

logistic regression classifier

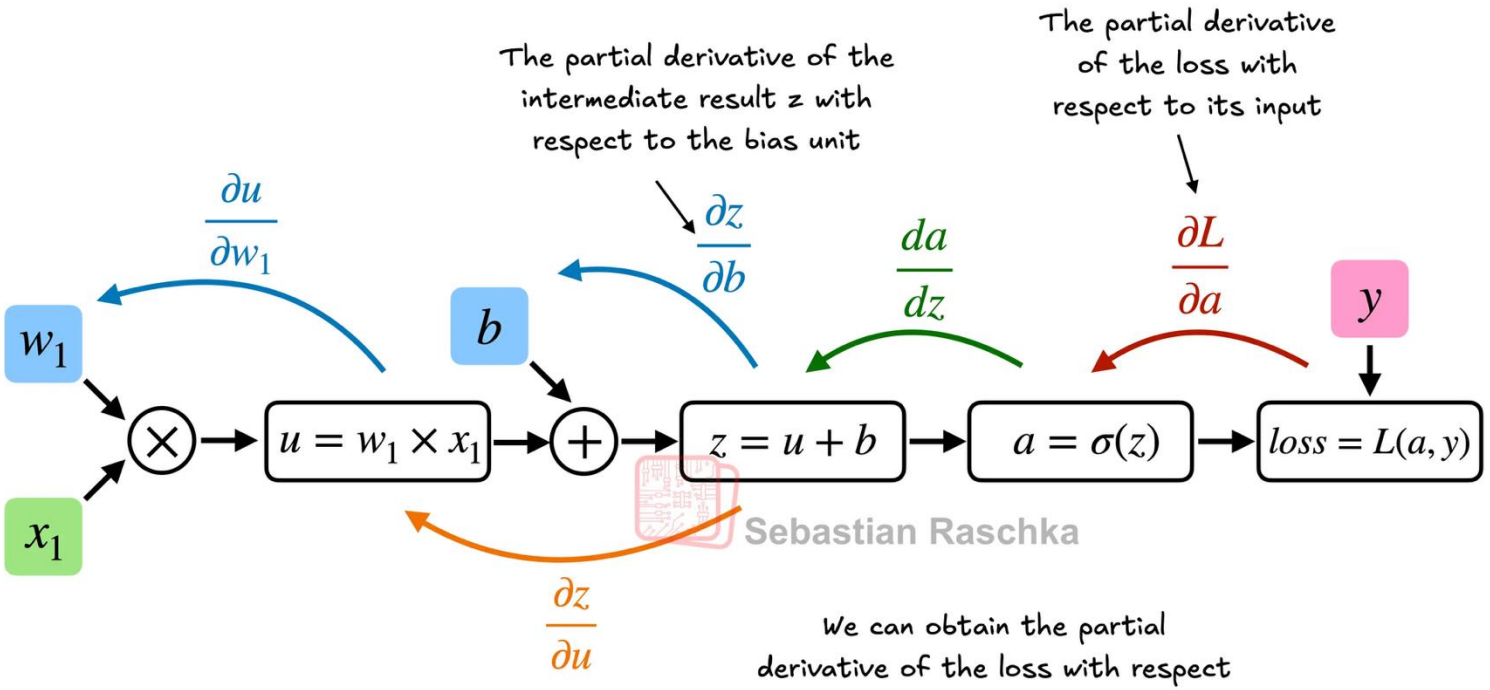
```
import torch.nn.functional as F

y = torch.tensor([1.0])  # true label
x1 = torch.tensor([1.1]) # input feature
w1 = torch.tensor([2.2], requires_grad=True)
) # weight parameter
b = torch.tensor([0.0], requires_grad=True)
)  # bias unit

z = x1 * w1 + b        # net input
a = torch.sigmoid(z)    # activation & output

loss = F.binary_cross_entropy(a, y)
print(loss)
```

Backpropagation

Main training algorithm for neural networks

Calculate gradients of a loss function with respect to the model parameters by applying chain rules

PyTorch's autograd provides functions to compute gradients in computational graphs automatically



We can obtain the partial derivative of the loss with respect to the trainable weight by chaining the individual partial derivative in the graph

$$\frac{\partial L}{\partial w_1} = \frac{\partial u}{\partial w_1} \times \frac{\partial z}{\partial u} \times \frac{da}{dz} \times \frac{\partial L}{\partial a}$$

$$\frac{\partial L}{\partial b} = \frac{\partial z}{\partial b} \times \frac{da}{dz} \times \frac{\partial L}{\partial a}$$

Similar to above, we can compute the partial derivative of the trainable derivative by applying the chain rule

```
loss.backward()
print(w1.grad)
print(b.grad)
```

COMPUTATION GRAPH

Lazy Execution Concepts

Eager Execution (PyTorch default)

- Operations execute immediately as called
- Each op returns a concrete tensor result
- Easy to debug
- Standard Python control flow works naturally

Lazy / Deferred Execution

- Builds a graph of operations first
- Actual computation deferred until needed
- Enables optimizations: operator fusion, reordering
- e.g. torch.compile

torch.compile() — Best of Both Worlds

```
@torch.compile                    # JIT-compiles the function on first call, Python decorator
def train_step(model, x, y):      # → builds optimized graph lazily
    pred = model(x)               # → fuses ops, eliminates redundancies
    loss = F.cross_entropy(pred, y)
    loss.backward()
    return loss
```

train_step = torch.compile(train_step) # equivalent to the decorator

Summary

Tensor Basics

Create, reshape, and manipulate multi-dimensional arrays with view, reshape, permute, transpose

Device Management

Move tensors between CPU and GPU with .to(), .cuda(), .cpu() — minimize transfers

GPU Performance

Use torch.cuda.synchronize() for accurate timing; GPU excels at large parallel ops

Computation Graphs

Autograd builds dynamic graphs during forward pass; .backward() computes all gradients automatically

Homework (Optional)

- Implement linear regression with gradient descent
  - Compare CPU vs. GPU performance with different number of features/sample sizes
  - Use autograd directly not torch.optim